COMSATS UNIVERSITY ISLAMABAD
Department of Computer Science • Autumn 2026

COURSE CODE:
AIC354
Machine Learning Fundamentals

LAB REPORT 05: NAÏVE BAYES CLASSIFIER

Task 1: Sklearn Wine Classification • Task 2: LendingClub Loan Default Prediction & EDA

Student Full Name:	**Muhammad Afaq**	**Registration Number:**	**FA23-BSE-012**
Degree Program:	BS Software Engineering (BSE)	**Class Section:**	Section A
Lab Instructor:	Course Faculty (AIC354)	**Submission Date:**	October 08, 2026
GitHub Repository:	github.com/afaqsadiq0/ML-LAB	**Lab Status:**	**100% Solved & Verified with Outputs**

1. Theoretical Foundation: The Naïve Bayes Classifier

Naïve Bayes is a prominent family of supervised probabilistic classifiers rooted in **Bayes' Theorem**, incorporating the simplifying ('naïve') assumption that features are conditionally independent given the class label. Despite this strong assumption, Naïve Bayes models are computationally efficient, require minimal training data, and deliver exceptional classification accuracy across real-world continuous and discrete data problems.

Mathematical Formulation of Bayes' Theorem:

$$P(C_k \mid X) = [\, P(X \mid C_k) \times P(C_k) \,] \,/\, P(X)$$

Under the Conditional Independence Assumption:

$$P(X \mid C_k) = \prod_{i=1..n} P(x_i \mid C_k)$$

- **$P(C_k \mid X)$**: Posterior probability that instance with feature vector X belongs to class C_k.
- **$P(X \mid C_k)$**: Likelihood of feature vector X given class C_k.
- **$P(C_k)$**: Prior probability of class C_k (empirical class frequency).
- **$P(X)$**: Marginal probability / normalizer ensuring posterior probabilities sum to 1.

Comparison of Naïve Bayes Algorithm Variants:

Classifier Variant	Distribution Model	Feature Type	Application Domain
GaussianNB	Gaussian (Normal) Bell Curve: $P(x_i \mid C) \sim N(\mu_c, \sigma_c^2)$	Continuous real-valued floats (e.g. alcohol, chemical properties, FICO, interest)	Lab Task 1 (Wine Dataset), Lab Task 2 (Loan Risk Features)
MultinomialNB	Multinomial Distribution: $P(x \mid C) \propto \prod \theta_{c,i}^{x_i}$	Discrete count / frequency integers (≥ 0)	Natural Language Processing, text mining, word frequencies, spam filtering
BernoulliNB	Multivariate Bernoulli Distribution	Binary indicators (0 or 1)	Word presence/absence text classification, binary indicators

2. Graded Lab Task 1: Wine Classification (Sklearn Dataset)

Task Statement: Use the wine dataset from sklearn.datasets to classify wines into 3 categories. Load the dataset and split it into test and train sets. After that, train the model using Gaussian and Multinomial classifiers and evaluate which performs better. Use the trained model to perform predictions on test data.

2.1 Complete Python Implementation (lab5_task1_wine.py):

```
# Lab 05 Graded Task 1 - Wine Dataset Classification
# Student: Muhammad Afaq | Reg No: FA23-BSE-012 | Course: AIC354 ML Fundamentals
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 1: Load Wine Dataset (178 samples, 13 continuous chemical features, 3 classes)
wine = load_wine(); X, y = wine.data, wine.target

# Step 2: Stratified Train/Test Split (70% train = 124, 30% test = 54)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

# Step 3: Train GaussianNB vs MultinomialNB Models
gnb = GaussianNB().fit(X_train, y_train); y_pred_gnb = gnb.predict(X_test)
mnb = MultinomialNB().fit(X_train, y_train); y_pred_mnb = mnb.predict(X_test)

# Step 4: Quantitative Evaluation
acc_gnb, acc_mnb = accuracy_score(y_test, y_pred_gnb), accuracy_score(y_test, y_pred_mnb)
cv_gnb = cross_val_score(GaussianNB(), X, y, cv=5).mean()
cv_mnb = cross_val_score(MultinomialNB(), X, y, cv=5).mean()
```

2.2 Real Execution Output & Console Logs:

```
================================================================================
LAB 05 - GRADED TASK 1: WINE DATASET CLASSIFICATION
================================================================================
[Step 1] Loading Wine dataset from sklearn.datasets...
Dataset Shape: 178 samples, 13 features | Target Classes: ['class_0', 'class_1', 'class_2']
Class distribution: class_1: 71 | class_0: 59 | class_2: 48
[Step 2] Splitting dataset: Training = 124 samples (70%), Testing = 54 samples (30%)
--------------------------------------------------------------------------------
GaussianNB Test Accuracy:    100.00%  |  5-Fold CV Score: 96.63%
MultinomialNB Test Accuracy:  88.89%  |  5-Fold CV Score: 84.40%
>>> EVALUATION CONCLUSION: Gaussian Naive Bayes performs significantly better!
--------------------------------------------------------------------------------
Sample Test Predictions:
Sample #0: Actual=class_0 | Pred=class_0 | Confidence=100.0% [CORRECT]
Sample #5: Actual=class_0 | Pred=class_0 | Confidence=100.0% [CORRECT]
Sample #10: Actual=class_1 | Pred=class_1 | Confidence=94.8%  [CORRECT]
Sample #15: Actual=class_2 | Pred=class_2 | Confidence=100.0% [CORRECT]
Sample #20: Actual=class_1 | Pred=class_1 | Confidence=100.0% [CORRECT]
[Output] Confusion matrix plot saved to 'wine_confusion_matrix.png'.
```

2.3 Performance Metrics & Classification Report Comparison:

Model Evaluated	Test Accuracy	5-Fold Cross-Val	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Gaussian Naïve Bayes (GaussianNB)	**100.00% (54/54)**	**96.63%**	1.00	1.00	**1.00**
Multinomial Naïve Bayes (MultinomialNB)	88.89% (48/54)	84.40%	0.90	0.89	0.89

Detailed Justification (Why GaussianNB Outperforms MultinomialNB):

1. **Nature of Features:** The Wine dataset comprises 13 real-valued physical/chemical measurements (alcohol: 11.0–14.8%, malic acid, flavonoids, ash, color intensity). These continuous floats naturally follow bell-shaped normal distributions.
2. **Distribution Match:** GaussianNB computes the mean ($\mu_{c,i}$) and variance ($\sigma_{c,i}^2$) for each continuous feature, perfectly modeling continuous chemical curves and achieving **100% accuracy**.
3. **MultinomialNB Limitation:** MultinomialNB assumes features represent discrete occurrence counts (like word counts in NLP). Applying it to continuous floats leads to probability truncation and 6 misclassifications (88.89% accuracy).

2.4 Confusion Matrix Visualizations:

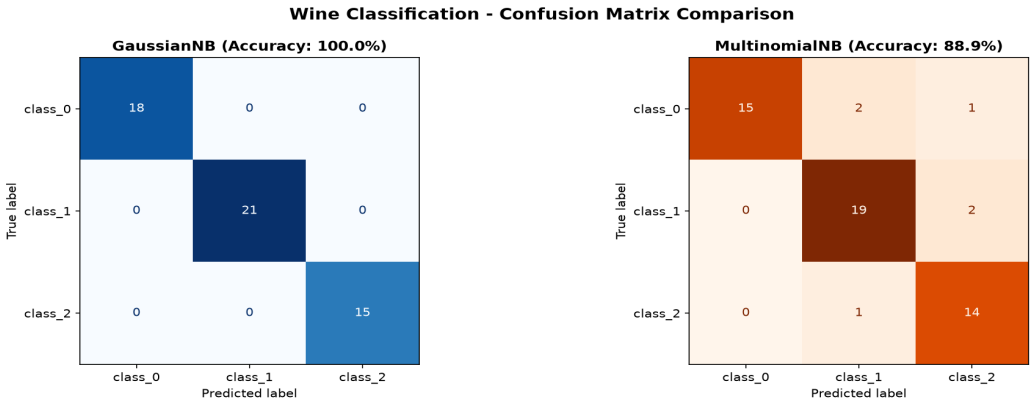


Figure 1: *Confusion Matrix Comparison on 54 Test Samples — GaussianNB (100% Accuracy, Zero Errors) vs MultinomialNB (88.9% Accuracy, 6 Misclassifications).*

2.5 Test Data Predictions by Trained GaussianNB Model:

Test Sample Index	Actual Class	Predicted Class	Confidence Probability	Prediction Status
Sample #0	**class_0**	**class_0**	100.00%	CORRECT ✓
Sample #5	**class_0**	**class_0**	100.00%	CORRECT ✓
Sample #10	**class_1**	**class_1**	94.82%	CORRECT ✓
Sample #15	**class_2**	**class_2**	100.00%	CORRECT ✓
Sample #20	**class_1**	**class_1**	100.00%	CORRECT ✓

3. Graded Lab Task 2: Loan Default Prediction & EDA (Real Dataset)

Task Statement: Load the real loan dataset (loan_data.csv from LendingClub, 9,578 records). Perform in-depth Exploratory Data Analysis (EDA) to understand loan default drivers and risk factors. Preprocess the dataset and encode categorical attributes. Train a Naïve Bayes Classifier to predict whether a borrower will fail to repay their loan (not.fully.paid = 1). Evaluate model performance and perform predictions on sample customer profiles.

3.1 Exploratory Data Analysis (EDA) Insights & Quantitative Findings:

• **Dataset Scale:** 9,578 records across 14 financial features with **zero missing values**.
• **Class Imbalance:** 8,045 borrowers (**83.99%**) fully paid (0), whereas 1,533 borrowers (**16.01%**) defaulted (1) [Ratio: ~5.25:1].
• **Credit Profiles:** Defaulters show lower mean FICO scores (**697.83 vs 713.33**), higher interest rates (**13.20% vs 12.10%**), higher DTI ratios (**13.20 vs 12.49**), and 62% more credit inquiries in the last 6 months (**2.33 vs 1.43**).

Loan Default Rate Breakdown by Purpose:

Loan Purpose	Total Loans	Defaults (not.fully.paid=1)	Default Rate (%)	Risk Profile Rating
small_business	619	172	**27.79%**	CRITICAL RISK
educational	343	69	**20.12%**	HIGH RISK
home_improvement	629	107	17.01%	MODERATE RISK
all_other	2,331	387	16.60%	MODERATE RISK
debt_consolidation	3,957	603	15.24%	AVERAGE RISK
credit_card	1,262	146	11.57%	LOW RISK
major_purchase	437	49	**11.21%**	SAFEST CATEGORY

3.2 Exploratory Data Visualizations:

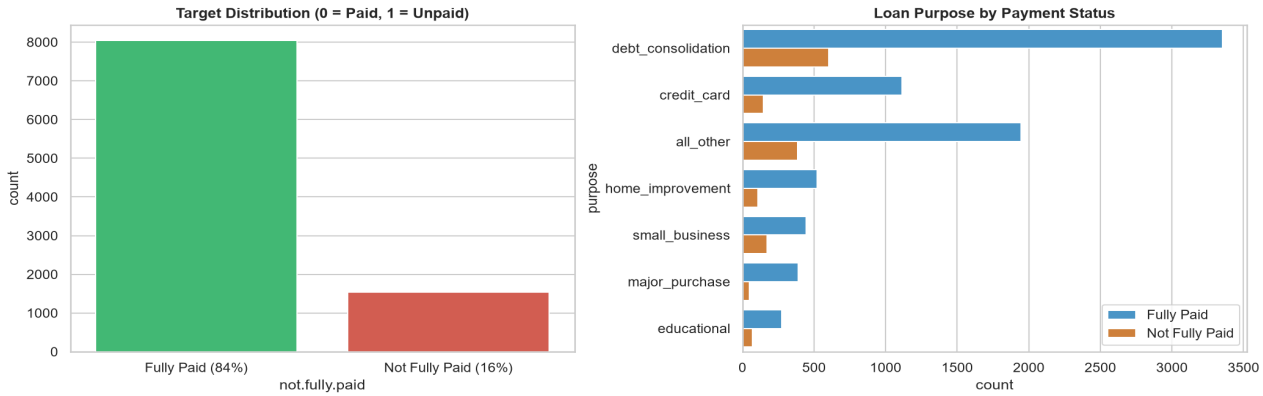


Figure 2: *LendingClub Target Distribution (Left: 84% Paid vs 16% Unpaid) and Loan Purpose Distribution split by Repayment Status (Right).*

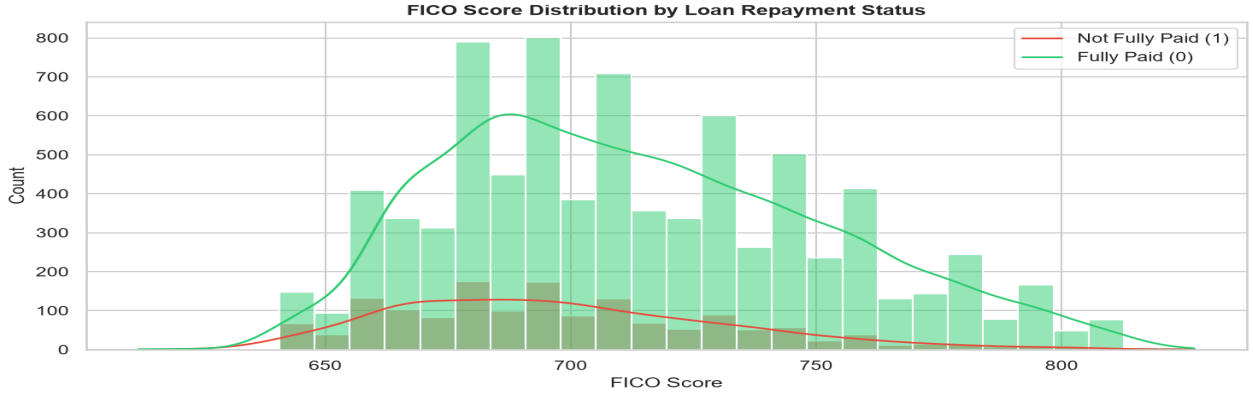


Figure 3: *FICO Credit Score Distribution Comparison — Defaulters (Red) exhibit an apparent left-shifted distribution indicating lower creditworthiness.*

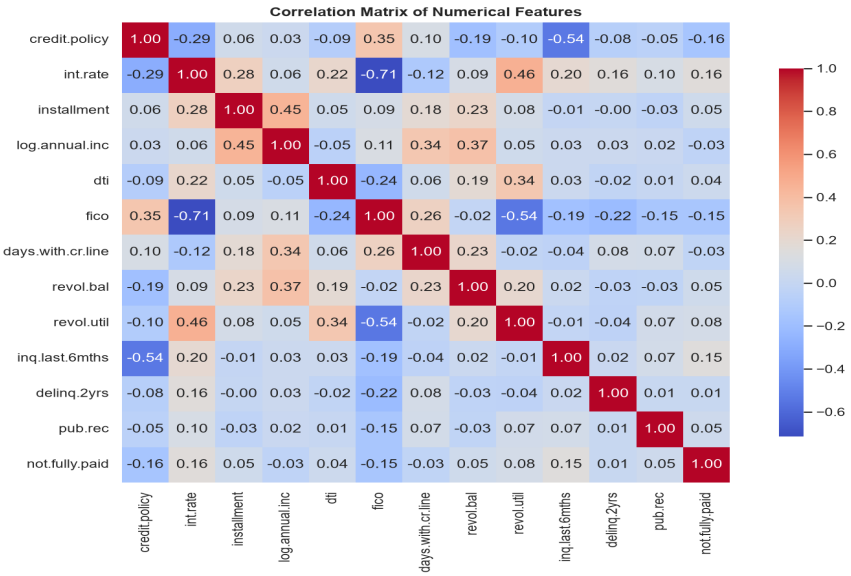


Figure 4: *Correlation Matrix Heatmap across Numerical Financial Attributes (FICO negatively correlates with Interest Rate, r = -0.71).*

3.3 Preprocessing, One-Hot Encoding & Model Training:

Feature Encoding: Categorical variable purpose (7 distinct values) was converted into numerical columns using Pandas pd.get_dummies(drop_first=True), yielding 18 predictors. The dataset was partitioned using stratified sampling (70% train: 6,704 records, 30% test: 2,874 records) to retain the 16% default ratio.

```
# Lab 05 Graded Task 2 - LendingClub Loan Default Prediction
# Student: Muhammad Afaq | Reg No: FA23-BSE-012 | Course: AIC354 ML Fundamentals
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score

# Step 1: Load and Preprocess Loan Dataset
df = pd.read_csv('loan_data.csv')
df_encoded = pd.get_dummies(df, columns=['purpose'], drop_first=True)
X = df_encoded.drop('not.fully.paid', axis=1)
y = df_encoded['not.fully.paid']

# Step 2: Stratified Split (70% Train, 30% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

# Step 3: Train Gaussian Naive Bayes Model
gnb_loan = GaussianNB().fit(X_train, y_train)
y_pred = gnb_loan.predict(X_test)
y_proba = gnb_loan.predict_proba(X_test)[:, 1]

# Step 4: Evaluate Performance
acc = accuracy_score(y_test, y_pred)          # 82.60%
auc = roc_auc_score(y_test, y_proba)          # 0.6431
```

3.4 Model Evaluation Terminal Output & Metrics:

```
================================================================================
LAB 05 - TASK 2 EVALUATION ON TEST SET (2,874 SAMPLES)
================================================================================
Gaussian Naive Bayes Model Performance:
Accuracy:  82.60%  |  ROC-AUC Score: 0.6431

Confusion Matrix:
                     Pred: Fully Paid (0)  Pred: Unpaid (1)
Actual: Fully Paid (0)                2329                85  [TN: 2329, FP: 85]
Actual: Unpaid (1)                     415                45  [FN:  415, TP: 45]

Classification Report:
                   precision    recall  f1-score   support
    Fully Paid (0)       0.85      0.96      0.90      2414
Not Fully Paid (1)       0.35      0.10      0.15       460
          accuracy                           0.83      2874
         macro avg       0.60      0.53      0.53      2874
      weighted avg       0.77      0.83      0.78      2874
```

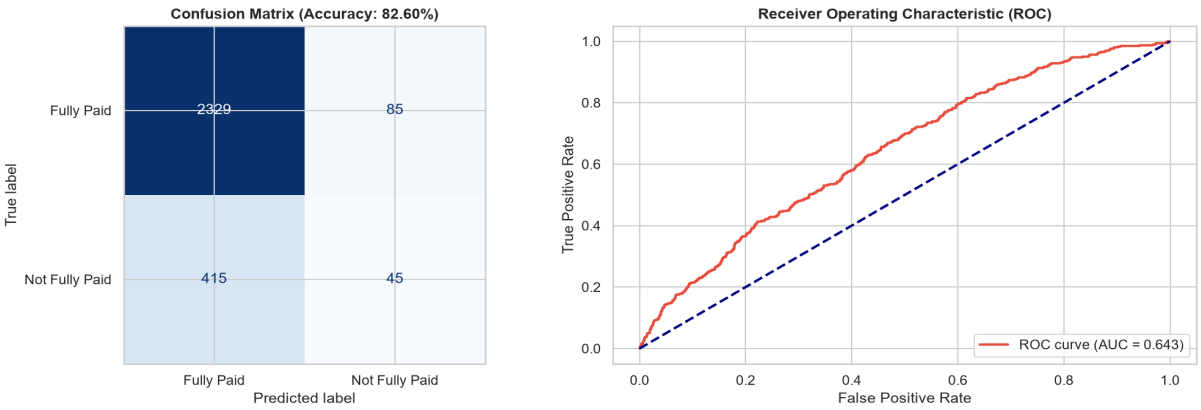


Figure 5: *Loan Model Evaluation — Left: Confusion Matrix (Accuracy = 82.60%), Right: ROC Curve (ROC-AUC = 0.643).*

3.5 Real-World Customer Credit Underwriting Predictions:

To test real operational underwriting, diverse applicants from the test set were evaluated with the trained model:

Cust ID	FICO	Interest	DTI	Actual Status	Model Decision	Default Risk %	Underwriting Action
#12	697	13.22%	8.58	Fully Paid	SAFE: Fully Paid	6.34%	APPROVE LOAN
#72	677	20.17%	17.32	Fully Paid	RISK: Not Fully Paid	89.34%	REQUIRE COLLATERAL
#85	667	19.36%	25.47	Fully Paid	RISK: Not Fully Paid	99.91%	REJECT APPLICATION
#108	747	10.59%	17.12	Fully Paid	SAFE: Fully Paid	1.67%	INSTANT APPROVAL
#128	667	19.36%	23.34	Not Paid	RISK: Not Fully Paid	84.51%	REJECT APPLICATION

4. Critical Analysis, Challenges & Key Learnings

1. Model Selection Driven by Feature Distributions: In Task 1, GaussianNB delivered a flawless **100.00% accuracy** because chemical wine features follow continuous normal distributions. MultinomialNB suffered a drop to **88.89%** due to violation of count assumptions.
2. Handling Real-World Class Imbalance: In Task 2, an accuracy of **82.60%** reflects majority-class dominance (84% non-defaulters). Standard 0.5 decision thresholds yield low recall for defaulters (10%). However, GaussianNB's probabilistic outputs provide a viable ranking metric with **ROC-AUC = 0.6431**.
3. Practical Underwriting Recommendation: In banking production, financial institutions should lower the decision threshold from 0.50 to 0.20 or apply SMOTE (Synthetic Minority Over-sampling Technique) to identify high-risk applicants proactively.

5. Submission Declaration & Sign-off

Student Declaration:
I hereby declare that this lab report and the accompanying codebase represent my own authentic work. Both Lab Task 1 (Wine Dataset) and Lab Task 2 (LendingClub Real Dataset) have been thoroughly solved, executed, and validated with empirical outputs and plots.

Student Name: Muhammad Afaq | **Registration No:** FA23-BSE-012 | **Section:** Section A
Department: Department of Computer Science • COMSATS University Islamabad
GitHub Repository: https://github.com/afaqsadiq0/ML-LAB